

Programación III :

Clase 1:

En la Programación Orientada a Objetos (POO) se buscan (o diseñan) **entidades independientes que colaboren** en la resolución de un problema, cada una de estas entidades tendrá un conjunto de habilidades propias y responsabilidades designadas (cada entidad es responsable de ciertas tareas). Para resolver un problema se utilizan un conjunto de estas entidades, las cuales interactúan entre sí, enviando mensajes entre ellas y proporcionando en cada mensaje la información necesaria para resolver la tarea y devolver una respuesta a quien le invoco.

Paradigma : es lo que se debe observar y escrutar, es el tipo de interrogantes que se supone hay que formular para hallar respuestas en relación al objetivo. Establece cómo deben estructurarse estos interrogantes. La POO es un paradigma de programación que usa objetos y sus interacciones para diseñar aplicaciones y programas. Está basado en varias técnicas, como por ejemplo: herencia, abstracción, polimorfismo y encapsulamiento.

¿Qué es un paradigma? : es un conjunto de elementos y reglas. Brinda un marco para el diseño y la construcción de Programas

Paradigma Estructurado	Paradigma Orientado a Objetos
● sistemas contienen datos y algoritmos ● los programas manipulan los datos ● se construyen en base a : ○ asignación ○ secuencia ○ repetición ○ condicional ● los programas están organizados por ○ descomposición funcional ○ flujo de datos ○ módulos	● los sistemas están compuestos por un conjunto de objetos ● los objetos son responsables de llevar a cabo ciertas acciones ● los objetos colaboran para llevar a cabo sus responsabilidades ● los programas están organizados en base a clases y jerarquías de herencia. ● los principios del paradigma OO según Alan Kay: ○ todo es un objeto ○ los objetos se comunican enviando y recibiendo mensajes ○ los objetos tienen su propia memoria (en término de objetos)

Objeto: es todo aquello que pueda ser identificable dentro de una especificación de requerimientos o problema y posea:

- **Características:** una manera de representar sus datos (atributos)
- **Comportamiento:** lo que puede hacer (correr, saltar, volar, etc)
- **Interacción / comunicación:** con otros objetos por medio de sus métodos.

Pueden ser agrupados por clases

Los objetos son instancias de una clase, es decir, las unidades de una clase específica que existe en el mundo real.

Clase: es un conjunto de objetos donde se determina el modelo que tendrá cada uno de estos objetos sin ser instanciados. Es una definición abstracta de un objeto, es decir, define su estructura y el comportamiento compartido por los objetos y sirve como modelo para la creación de objetos. Es una descripción de un grupo de objetos con:

- propiedades en común (atributos)
- comportamientos similares (operacione)
- la misma forma de relacionarse con otros objetos (relaciones)

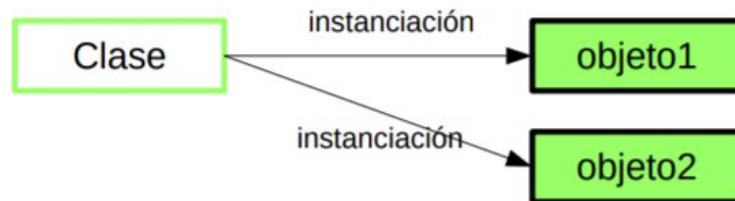
- una semántica en común (significan lo mismo)

Una clase es la abstracción de las propiedades y operaciones o acciones que ejecutan los objetos del mundo real

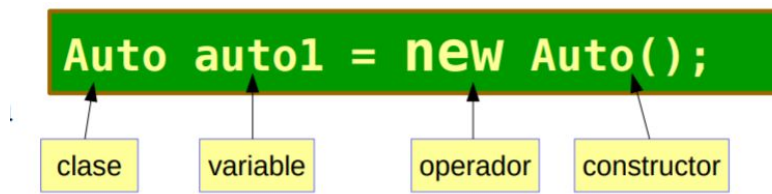
(*1)

Instanciación:

El mecanismo para la **creación de objetos** a partir de la **clase** se llama **instanciación de la clase**



¿Cómo instanciar una clase y así crear un nuevo objeto? : utilizando el operador **new**...



involucra los siguientes pasos:

1. se aloja espacio en memoria para la variable y para el objeto.
2. se inicializan los atributos del objeto con los valores por defecto.
3. se ejecuta el constructor.
4. se asigna la referencia del nuevo objeto a la variable.

Estado de un objeto: hace referencia al conjunto de valores que contienen sus atributos en un instante dado de tiempo. Está compuesto por las **variables de instancia** del objeto, (**atributos**), las cuales pueden hacer referencia a:

- Propiedades del objeto (marca, color, tamaño)
- Otros objetos con los cuales pueden colaborar para llevar a cabo sus responsabilidades (Persona, Auto)

Todos sus atributos es conveniente que sean privados del objeto, y así ningún otro objeto pueda acceder a ellos.

Variable : es una unidad básica de almacenamiento donde cada una tiene su tipo de dato, identificador y ámbito.

Arrays:

```

//Declaraciones juntas
Persona[] arraypersona = new Persona[10];
Persona p1 = new Persona(10,"Maria", "Pedro");
arraypersona[1] = p1;

//Declaraciones unidas
Persona[] arraypersona = new Persona[10];
arraypersona[2] = new Persona(10,"Maria", "Pedro");
  
```

Variable de instancia o atributos: son las características individuales que diferencian a un objeto de otro. Los atributos se guardan en variables denominadas de instancia y cada objeto en particular puede tener valores distintos para estas variables.

¿Qué hace un objeto?: un objeto se define en término de su comportamiento. El comportamiento indica qué sabe hacer el objeto (determina cuales son sus responsabilidades). Se especifica a través del conjunto de mensajes que el objeto sabe responder: **protocolo**

¿Como lo hace?: la implementación de un objeto se especifica a través de un conjunto de métodos. Cuando un objeto recibe un mensaje, responde activando el método asociado. Es privado del objeto : el que envía el mensaje delega en el receptor manera de resolverlo, sin importarle cómo lo resuelve

Interacción entre Objetos usando mensajes: el comportamiento de un objeto está definido en términos de los mensajes que este entiende.

1. Para poder enviarle un mensaje a un objeto, primero hay que conocerlo
2. Al enviarle un mensaje a un objeto, este responde activando el método asociado a ese mensaje (siempre y cuando exista)
3. Como resultado del envío de un mensaje, puede retornar un objeto.
4. Se especifica con el nombre correspondiente al protocolo del objeto receptor
5. Se indica cuales son los parámetros, es decir, la información necesaria para resolver el mensaje

Se utiliza la siguiente sintaxis:

```
<objeto receptor>.<nombre del mensaje>(<parámetros>);
```

21

Ej:

Decirle a una cuenta bancaria que deposite \$100 se escribe como:

```
CajaDeAhorro unaCuenta = new CajaDeAhorro();  
unaCuenta.depositar(100);
```

Métodos: expresan la forma de llevar a cabo la semántica propia de un mensaje particular.

Puede realizar 3 cosas:

1. Modificar el estado interno del objeto receptor
2. Colaborar Con otros objetos
3. Retornar y terminar

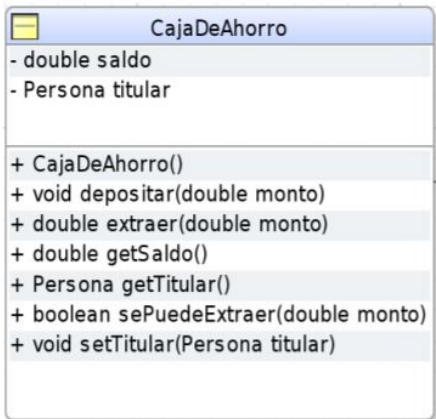
El **pasaje de parametros** en Java **es por valor**. Por lo cual se debe de tener en cuenta el caso de que el parámetro es de tipo Objeto, ya que el valor que contiene es una referencia a un Objeto y por lo tanto, se puede alterar el estado del objeto referenciado. En el momento de la invocación, el parámetro formal copia el contenido del parámetro real (*pasaje de parámetros por valor*), pero lo que está copiando en caso de ser un Objeto, es la referencia a este que contiene el parámetro real. Osea, cualquier alteración (dentro del método) del estado del objeto referenciado por el parámetro formal impacta sobre el objeto.

(*1)

Clases : se especifican por medio de un nombre, el estado interno y los métodos asociados que definen su comportamiento

- **Nombre**: comienza con mayúscula y no posee espacios

- **Atributos:** minúscula y sin espacios
- **Métodos:** para cada uno se debe especificar como mínimo el nombre, parámetros y retorno. Comienzan siempre con minúscula



Constructor: se utiliza para realizar inicializaciones más complejas. Es un bloque de sentencias que se pueden utilizar para inicializar un objeto antes de que se devuelva la referencia a él mediante el new(...). Tiene el mismo nombre que la clase. Puede admitir cero o más parámetros. No es un método por lo que no tiene retorno. Se invoca después de que las variables de instancia del objeto de nueva creación tengan asignado sus valores iniciales por defecto y después de que sus inicializadores explícitos se hayan asignado.

Abstracción: es el proceso de supresión de detalles respecto de un fenómeno, entidad o concepto. El objetivo es centrarse en los aspectos más significativos.

UML Diagramas de Clase: en este :

- ★ una clase se describe en función de sus atributos y mensajes. Notación



Clase II

Propiedades de los objetos:

- ★ **Abstracción:** significa extraer las propiedades esenciales de un objeto que lo distinguen de los demás tipos de objetos y proporciona fronteras conceptuales definidas respecto al punto de vista del observador
- ★ **Encapsulamiento:** es la propiedad que permite ocultar al mundo exterior la representación interna del objeto. Esto quiere decir que el objeto puede ser utilizado, pero los datos esenciales del mismo no son conocidos fuera de él.
- ★ **Herencia:** es la propiedad que permite a los objetos construirse a partir de otros objetos. La clase base contiene todas las características comunes. Las subclases contienen las características de la clase base más las características particulares de la subclase. Si la subclase hereda características de una clase base, se trata de herencia simple. Si hereda de dos o más clases base, herencia múltiple.
- ★ **Polimorfismo:** significa “cualidad de tener más de una forma”. En POO, se refiere al hecho de que una misma operación puede tener diferentes objetos. En otras palabras, diferentes objetos reaccionan al mismo mensaje de modo diferente.

Modos de acceso: es la forma de representar el **encapsulamiento**

1. **Público (+):** atributos o métodos que son accesibles fuera de la clase. Pueden ser llamados por cualquier clase, aun si no está relacionada a ella.

2. **Privado (-):** atributos o métodos que son solo accesibles dentro de la implementación de la clase
3. **Protegido (#):** atributos o métodos que son accesibles para la propia clase y sus clases hijas (subclases).

Sobrecarga de Métodos: dos o más métodos pueden tener el mismo nombre si tienen diferente número o tipo de parámetros, y por lo tanto diferente signatura. El nombre del método tiene más de un significado.

No se puede sobrecargar un método solamente modificando el retorno.

Los constructores también pueden sobrecargarse.

*signatura: consiste del nombre y parámetros (números y tipos). Es toda cabecera excepto el retorno.

Static:

- Atributos Estáticos: son atributos compartidos por todos los objetos de una clase. Los atributos específicos de clase se obtienen declarándose como **static**. No se necesita crear un objeto para poder acceder a estos atributos. Se puede acceder invocando el nombre de la clase seguida del operador de selección (.) y luego el nombre del campo
- Métodos Estáticos: se invoca en nombre de una clase completo, no sobre una instancia. Puede acceder solo a campos estáticos y otros métodos estáticos de la clase. Dentro de un método estático no hay referencia a *this* (no existe el objeto)

Constantes: en Java se definen utilizando la palabra clave **final**. Una vez que un atributo declarado final es inicializado, no es posible cambiar su valor. Existen dos formas de inicializarlo:

- Cuando lo declaramos: `private final int var = 12;`
- Luego de declararlo, puede ser dentro de un método: `private final int var; .. this.var = 12;`

Si luego de inicializar el valor del atributo constante decidimos modificarlo, se producirá un error, ya que no puede cambiar su valor.

¿Qué sucede si queremos declarar un objeto constante?: no puedo cambiar la dirección de memoria del mismo, pero si cambiar los atributos que contiene este (siempre y cuando estos no sean constantes)

Garbage Collection / Destructor de objetos: el manejo manual de memoria es una tarea compleja y propensa a errores. Cuando no hay ninguna referencia hacia un objeto, este se marca para ser recogido por el recolector. Los lenguajes OO usualmente incluyen un mecanismo llamado Garbage Collection el cual:

- detecta los objetos que ya nadie conoce y libera la memoria que ocupaban.
- es automático.
- es seguro.
- es transparente, corre en background.

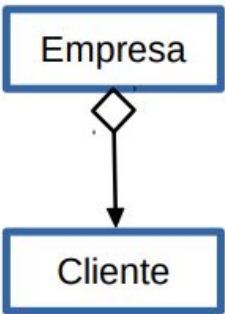
Tiene como desventaja:

- no puede utilizarse en sistemas de tiempo real.
- los primeros garbage collectors tenían un overhead importante.

Finalize: este método se llama cuando va a ser liberada la memoria (que ocupa el objeto) por el recolector basura (garbage collector). Normalmente no es necesario redefinir este método en las clases.

Relación de Agregación: es un tipo de asociación que indica que una clase es parte de otra clase (**composición débil**). *Los componentes pueden ser compartidos por varios compuestos* (de la misma asociación de agregación o de varias asociaciones de agregación distintas). La

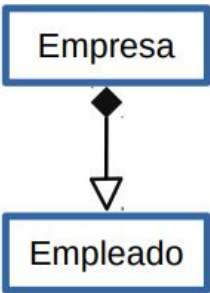
destrucción del compuesto no lleva a la destrucción de los componentes. Habitualmente se da con mayor frecuencia que la composición.



- tenemos una clase empresa
- tenemos una clase cliente
- una empresa agrupa a varios clientes

La agrupación se representa en UML mediante un diamante de color blanco colocado en el extremo en que está la clase que representa el “todo”

Relación de Composición: es una forma fuerte de asociación donde la vida de la clase contenida debe coincidir con la vida de la clase contenedor. *Los componentes constituyen una parte del objeto compuesto. De esta forma los componentes o pueden ser compartidos por varios objetos compuestos.* La supresión del objeto compuesto conlleva la supresión de los componentes.



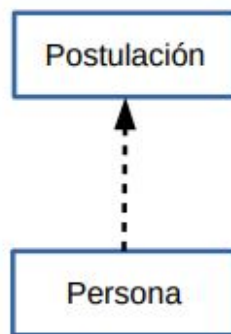
- tenemos una clase empresa
- un objeto empresa está a su vez compuesto por uno o varios objetos de tipo empleado.
- el tiempo de vida de los objetos empleado depende del tiempo de vida de empresa, ya que si no existe una empresa no pueden existir sus empleados.

El símbolo de composición es un diamante de color negro colocado en el extremo en el que está la clase que representa el “todo” (Compuesto)

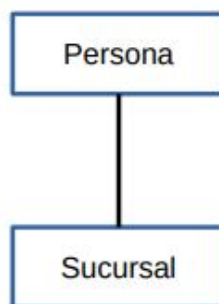
	AGREGACIÓN	COMPOSICIÓN
Varias asociaciones comparten los componentes	Sí	No
Destrucción de los componentes al destruir el compuesto	No	Sí
Cardinalidad a nivel de compuesto	Cualquiera	0..1 ó 1
Representación	Rombo transparente	Rombo negro

Relacion de Dependencia: La clase Persona tiene una relacion con dependencia con Postulación. Conceptualmente esto significa que la postulación es un objeto que la persona utiliza para algún fin, dentro de alguna operación que ella realice (por ejemplo postularse a un cargo). Pero una persona no tiene en su interior una postulación (como atributo), sino que solo

lo utiliza para realizar ciertas operaciones. Por ejemplo, un método de Persona que cree y retorne un objeto de tipo Postulación.



Relación de Asociación: en un diagrama de clases implica transitividad y bidireccional de clases. Por ejemplo una persona tiene como atributo interno a una sucursal, pero una sucursal también tiene un atributo de tipo persona, la cardinalidad de la asociación indicará si sucursal tiene una o muchas instancias de persona, con o cual en realidad el atributo de sucursal podría ser una lista de personas. Otra característica fundamental es que la vida de las instancias de ambas clases no dependen una de la otra.



Bloques de inicialización: se utilizan para realizar inicializaciones más complejas. Es un bloque de sentencias que aparece dentro de una declaración de la clase y fuera de la declaración de cualquier miembro o constructor, y que inicializa los campos del objeto. Se utiliza en inicializaciones no triviales, cuando no se necesitan parámetros de construcción y hay una razón para no proporcionar un constructor sin argumentos.

Singleton: es una única instancia de una clase. Una vez que se crea la instancia, no se puede crear otro.

Clase III

Herencia: la programación orientada a objetos permite a las clases expresar similitudes entre objetos que tienen algunas características y comportamiento común. Estas similitudes pueden expresarse usando **herencia**. El término **herencia** se refiere al hecho de que una clase **hereda** la características (atributos y variables) y el comportamiento (métodos) de otra clase.

- Los atributos heredados no deben volver a escribirse en la clase hija, excepto que se quiera **ocultar** al atributo de la clase padre.
- Los métodos heredados no deben volver a escribirse en la clase hija, excepto que se quiere **sobreescibir**

La herencia facilita la creación de objetos a partir de otros ya existentes e implica que una subclase obtiene todo el comportamiento (métodos) y eventualmente atributos (variables) de su superclase.

Aspectos:

- **Herencia de contrato o de tipo:** la subclase adquiere el tipo de la subclase y se puede utilizar polimórficamente allí donde se pudiera utilizar la superclase

- **Herencia de implementación:** la subclase adquiere la implementación de la superclase en términos de sus campos y métodos accesibles.

Una subclase puede agregar características y comportamiento y **reemplazar** o modificar el **comportamiento** heredado.

Como se indica la relación de herencia en Java?: con la palabra **extends**

```
//Código de la clase profesor, subclase
public class Profesor extends Persona
{
    //Campos específicos de la subclase
    private String IdProfesor;
    //Constructor de la subclase: incluye
    public Profesor(String nombre, String apellidos, int edad)
    {
        super(nombre, apellidos, edad);
        IdProfesor = "Unknown";
    } //Cierre del constructor
}
```

La clase **Java.lang.Object** es la raíz de la jerarquía de clases en Java. Cualquier clase que no especifique un padre directo, será subclase directa de Object.

Java permite múltiples niveles de herencia, pero la herencia es simple

Sobreescritura de Métodos: permite definir un método en una subclase, que tenga exactamente la misma firma que un método de la superclase. El objetivo de sobreescribir un método es cambiar su implementación.

Ocultamiento de Variables:

```
class Animal{
    float peso;
}

class Mamifero extends Animal{
    float peso;
    alimentoConsume(){
        float peso;
        ...
        peso= ...;
    }
}
```

Oculto la variable de instancia

18

Super: la palabra reservada super puede utilizarse para invocar a un miembro de la superclase oculto (atributo) o sobreescrito (método o constructor) por otro de la propia clase.

Manera de invocarlo:

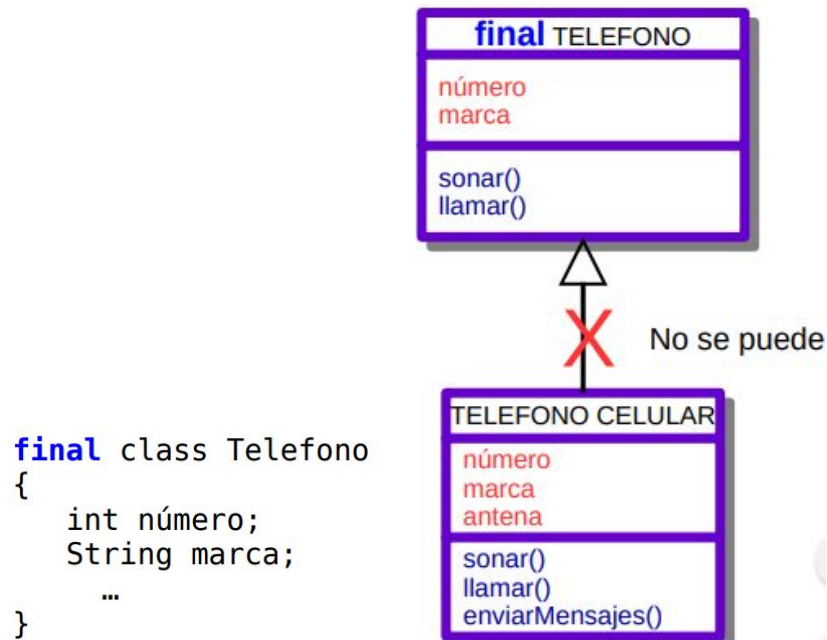
- Constructor del padre: super()
- Atributo del padre : super.id
- Método del padre : super.detalles()

Constructor: el constructor de una clase extendida puede tratar con variables de estado heredadas, pero solo su superclase sabe cómo inicializar correctamente su estado, de forma que cumpla su contrato. Los constructores de las clases hijas deben delegar la construcción del estado heredado (atributos del padre no sobrescritos), al constructor de su padre.

Si un constructor de la clase hija va a invocar a un constructor de la clase padre, la invocación SI O SI debe de estar en la primera línea del constructor de la clase hija

Especificadores de acceso: cuando sobreescribimos un método, podemos modificar la cabecera, pero podemos modificar el especificador de acceso, pero esto tiene ciertas reglas. Puedo cambiar el especificador de acceso si y sólo si este será más accesible, no al revés

Clase FINAL: si definimos una clase final esto significa que la clase no puede extenderse (tener hijos). Por lo tanto, es el punto final de la cadena de clases derivadas.



Metodo FINAL: esto significa que no puede redefinirse en las clases hijas.

```
class Telefono
{
    ...
    public final void sonar()
    {...}
}
```

No se puede :

```
class TelefonoCelular extends Telefono
{
    ...
    public final void sonar()
    {...}
}
```

Variable FINAL: para una variable, declararla como final hace referencia a que no puede ser redefinida en una clase derivada (hija), y que su valor no puede ser cambiado en ningún sitio, es decir, es constante. Al declarar una variable como final, se previene que su contenido sea modificado, sin embargo, Java permite diferenciar la definición de la inicialización. Si bien se parece a lo que es una constante, no es necesariamente una constante, ya que podría tener su valor inicial delegado al constructor, o a tiempo de ejecución, teniendo así en cada ejecución un valor diferente, constante, pero diferente en cada ciclo de vida.

Delegación: es la situación en la que una clase contiene (como atributos) una o más instancias de otra clase, a las que delegan parte de sus funcionalidades. Esta relación entre clases suele ser la más indicada cuando es necesaria una asociación entre las clases pero el principio de Liskov no se cumple.

Es una manera de crear clases, donde alguno de sus atributos pueden ser otros objetos permitiendo así las referencias. Las clases delegan algunas partes de sus comportamientos. Cada atributo, compone al objeto.

Diferencia entre Delegación y Herencia: cuando no se que hacer, opto primero por la delegación, ya que esta es una utilización selectiva, mientras que la herencia es un todo o nada.

Clase Concreta: se puede pensar en objetos concretos con características propias.

Clase Abstracta: el objetivo de una clase abstracta es ser extendida por clases concretas y definir una interfaz de comportamiento común de los objetos de sus subclases. Se utiliza la palabra **abstract**. Una clase abstracta representa un conjunto de atributos y métodos pero no tiene valores, no puede ser instanciada. Sus métodos se deben de sobrescribir, solamente en la clase abstracta existirá la cabecera con los parámetros. **Una clase abstracta tiene que tener al menos un método abstracto**

```
public abstract class Cuenta
{
    protected double saldo;
    private String titular;
    private int numCuenta;
```

```
abstract void extraer(double importe);
```

Especialización vs Generalización: una jerarquía de clases puede surgir de dos maneras

- **Generalización:**
 - diseñar clases por separado
 - descubrir características comunes y específicas
 - abstraer características comunes en una nueva clase
 - generar la jerarquía de clases usando la nueva clase como superclase de las clases previamente generadas
 - eliminar las características comunes de las subclases
 - verificar que la jerarquía respete el principio “es un”
- **Especialización:**
 - antes de diseñar una clase, analizar primero las clases existentes en el sistema
 - buscar una clase que tenga características comunes con la que estoy por diseñar
 - si la relación entre ambas clases respeta el principio “es un”, generar una nueva clase a partir de la clase existente.

Principios Solid:

SOLID

Inicial	Significado (acrónimo)	Concepto
S	SRP	Principio de Única Responsabilidad (Single responsibility principle) la noción de que un objeto solo debería tener una única responsabilidad.
O	OCP	Principio Abierto/Cerrado la noción de que las "entidades de software ... deben estar abiertas para su extensión, pero cerradas para su modificación".
L	LSP	Principio de sustitución de Liskov la noción de que los "objetos de un programa deberían ser reemplazables por instancias de sus subtipos sin alterar el correcto funcionamiento del programa".
I	ISP	Principio de Segregación de la Interfaz (Interface segregation principle) la noción de que "muchas interfaces cliente específicas son mejores que una interfaz de propósito general."
D	DIP	Principio de Inversión de Dependencia (Dependency inversion principle) la noción de que uno debería "Depender de Abstracciones. No depender de implementaciones." La Inyección de Dependencias es uno de los métodos que siguen este principio.

Clase IV

Principio de Sustitución de Liskov: “debe ser posible utilizar cualquier objeto instancia de una subclase en lugar de cualquier objeto instancia de su superclase sin que la semántica del programa escrito en los términos de la superclase se vea afectado”

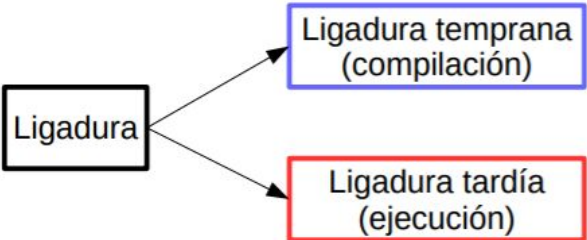
Polimorfismo: significa que proviene varias formas diferentes. Existen 3 tipos

- **Polimorfismo de Sobrecarga:** permite definir operadores cuyos comportamientos varían de acuerdo a los parámetros que se les aplican. Por ejemplo, el operador + entre dos enteros suma, y entre dos strings los concatena.
- **Polimorfismo Paramétrico:** es la capacidad para definir varios métodos utilizando el mismo nombre, pero usando parámetros diferentes (nombre y/o tipo). Selecciona Automáticamente el método correcto a aplicar en función del tipo de datos pasados en el parámetro.

```
int addition(int,int) devolvería la suma de dos números enteros.  
float addition(float, float) devolvería la suma de dos flotantes.  
char addition(char, char) daría por resultado la suma de dos caracteres
```

- **Polimorfismo de Inclusión / redefinición / subtipado:** es la habilidad para redefinir un método en clases que se hereda de una clase base. Por lo tanto, se puede llamar un método de objeto sin tener que conocer su tipo intrínseco. Intervienen al menos dos clases, la padre y la hija, en la hija se redefine el método que tiene la misma cabecera, pero su cuerpo es distinto, o el modificador de acceso (protected,private,public), puede hacerse más accesible, no menos.

Concepto de Enlace Tardío (Late Binding): la herencia permite el tratamiento del objeto como si fuera de su propio tipo o del tipo base. Esto permite que varios tipos (derivados de un mismo tipo base) sean tratados como si fueran uno solo. Cualquiera sea el tipo de clase que tenga, siempre voy a poder alojarlo en una clase de tipo padre / abuelo.



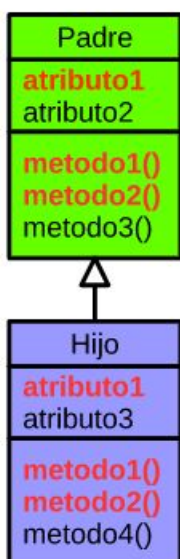
Ligadura: conexión de una llamada a un método.
Ligadura temprana: ej una inicialización

Acceso a miembros sobrescritos: al invocara un método mediante una referencia de objeto, la clase real del objeto gobierna la implementación que se utiliza. Cuando se accede a un campo se utiliza el tipo declarado de la referencia.

Ejemplo genérico:

```
Padre p = new Hijo();
```

p solo conoce los atributos de padre. No voy a poder hacer referencia a atributo 3.
p solo puede llamar a los métodos sobreescritos o heredados, pero no los nuevos. Nunca va a poder llamar a método 4, porque Padre no conoce a ese método, no existe dentro de su código.



Supongamos una jerarquía de clases que contemple la redefinición de algunos métodos en la clase hijo:

- **atributo1** de Padre es ocultado por atributo1 de Hijo
- **metodo1()** y **metodo2()** de Padre son sobreescritos en Hijo
- **atributo2** es propio de Padre
- **atributo3** es propio de Hijo
- **metodo3()** es heredado por Hijo
- **metodo4()** es propio de Hijo

Protected: un miembro protected puede ser accedido por las clases que extiendan de la clase actual, por clases del mismo paquete, y además puede ser accedido desde otra clase a través de referencias a objetos que sean al menos del mismo tipo que la clase (es decir, referencias del tipo de la clase o de uno de sus subtipos)

Ejemplo

Consideremos la implementación como **lista enlazada** de una fila. Los nodos de la fila están formados por objetos de tipo **Nodo**

Concepto de Contrato: para garantizar la aplicación del principio de sustitución de Liskov no cause problema, se debe de documentar el contrato de forma que simplemente con respetarlo se garantice que se pueda aplicar el principio. Se utilizan precondiciones y postcondiciones. En los métodos sobreescritos se deben respetar pre post. Se pueden relajar las pre y fortalecer las post.

Un contrato es el acuerdo establecido para el buen uso de esa clase o método. No siempre se puede codificar de manera tan robusta para evitar el mal uso de esa clase o método, por consiguiente se crean los contratos.

Contrato de un método: este tema está estrechamente relacionado con el proceso de asignar responsabilidades a las clases. Esta suma de suposiciones y compromisos son las que se

integran en los contratos, de manera que debemos aprender a documentarlas, a leerlas y a manejar los errores que se pueden producir cuando estos contratos no se cumplen. El contrato de un método establece bajo qué condiciones el método tendrá éxito y cuál será el resultado una vez que se termine su ejecución. Por ejemplo:

```
public void afiliarSocio(String cédula, String nombre)
    throws Exception
```

Podemos establecer que las suposiciones **ANTES** (condiciones previas) de ejecutar el método son:

- La lista de socios ya fue creada.
- La cédula no es null ni vacía.
- No se ha verificado si ya existe un socio con esa cédula.
- El nombre no es null ni vacío.

DESPUÉS (condiciones posteriores) de ejecutar el método, el resultado debe ser uno de los siguientes:

- Todo funcionó bien y el socio se afilió al club.
- Se produjo un error y se informó del problema con una **excepción**. El socio no quedó afiliado al club.

Precondición: conjunto de suposiciones, expresadas como condiciones que deben ser verdades para que el método se ejecute con éxito. Estas precondiciones pueden referirse a:

- el estado del objeto que va a ejecutar el método (el valor de sus atributos).
- el estado de algún elemento del mundo con el cual el objeto tenga una asociación.
- condiciones sobre los parámetros de entrada entregados al método.

Postcondición: es la descripción del resultado obtenido después de ejecutar un método. Está se expresa en terminos de un conjunto de condiciones que deben ser verdaderas después de que el método ha sido ejecutado, siempre y cuando no se haya lanzado una excepción. Estas precondiciones hacen referencia a:

- una descripción del valor de retorno.
- una descripción del estado del objeto después de haber ejecutado el método.

¿?

¿Un método debe verificar en algún punto las condiciones que hacen parte de la precondición?

: no, lo que aparece en la precondición se debe suponer como cierto y se debe utilizar como si lo fuera. Si algo falla en la ejecución por culpa de eso, es el problema de aquel que hizo la llamada sin cumplir el contrato.

¿Qué lugar ocupan las excepciones en los contratos? : un contrato solo debe decir que lanza una excepcion cuando, aun cumpliendose todo lo pedido en la precondición, es imposible llegar a cumplir la postcondición. Eso quiere decir que ninguna excepción puede asociarse con el incumplimiento de una precondición.

¿Que incluir en la precondición? : solo se deben incluir condiciones que resulten fáciles de garantizar por parte de aquel que utiliza el método.

¿Por que es conveniente verificar todo dentro del método invocado? : por eficiencia. Es mucho mejor repartir las responsabilidades de verificar las cosas entre el que hace el llamada y el que hace el método. Si en el contrato queda claro quien se encargará de que, es más fácil y eficiente resolver los problemas.

Documentación con Javadoc:



Objetivo: Mostrar un contrato completo y la página html generada por la herramienta Javadoc.

En este ejemplo se presenta el contrato del método de la clase Club que afilia un nuevo socio. En la parte de abajo aparece la visualización del archivo html generado automáticamente por la herramienta Javadoc.

```
/**
 * Este método afilia un nuevo socio al club.<br>
 *
 * <b>pre:</b> La lista de socios está inicializada (no es null).<br>
 * <b>post:</b> Se ha afiliado un nuevo socio en el club con los datos dados.<br>
 *
 * @param cedula Es la cédula del nuevo socio. cedula != null, cedula != ""
 * @param nombre Es el nombre del nuevo socio. nombre != null, nombre != ""
 *
 * @throws Exception si un socio con la misma cédula ya estaba afiliado al club,
 *                    dispara una excepción indicando que la nueva afiliación no se
 *                    pudo llevar a cabo.
 */
public void afiliarSocio( String cedula, String nombre ) throws Exception
{
}
```

Requerimientos: formato, ej

Nombre	R1 - Registrar una persona autorizada por un socio
Resumen	Los socios tienen un conjunto de personas autorizadas que pueden ingresar al club y hacer consumos en sus restaurantes. Este requerimiento permite a un socio registrar una de estas personas.
Entradas	
Nombre	Descripción
nombre	Nombre de la persona autorizada por el socio
cedula	Cédula del socio al que se registrará el autorizado
Resultados	
Nombre	Descripción
Registro de autorizado	El socio tiene una nueva persona autorizada

En nombre iria la cabecera del método

Asignación de Responsabilidades:

- Técnica del Experto: define quien es responsable de hacer algo, pero son las reglas del mundo las que nos dicen cómo cumplir con dicha responsabilidad.
- Técnica de Descomposición de Requerimientos: (ver filmina)

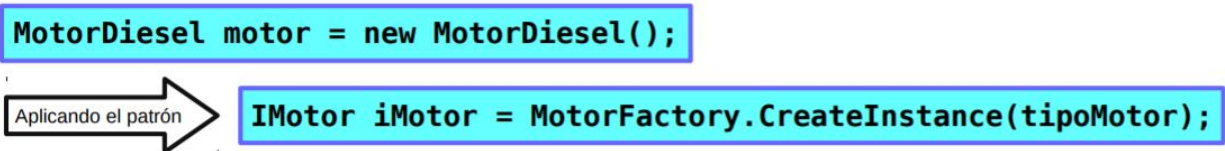
Excepciones: (ver filminas)

Patrones de diseño: son soluciones preestablecidas a problemas comunes. Existen varios tipos de patrones, ya que los problemas se pueden clasificar de maneras diferentes.
def: cada patrón describe un problema que ocurre una y otra vez en nuestro entorno y describe también un núcleo de la solución al problema, de forma que puede utilizarse un millón de veces sin tener que hacer dos veces lo mismo.

Un patrón de diseño es una descripción de clases y objetos comunicándose entre sí, adaptada para resolver un problema de diseño general en contexto particular

Patrón de diseño Factory: es de tipo creacional. Se crea un objeto sin exponer la lógica de la creación al cliente, y retorna una referencia al nuevo objeto creado, usando una interfaz común. Este tipo de patrón es aquel en el que se delega la instanciación de un objeto en otro, en lugar de recurrir a un simple new(). Esta forma de trabajar puede ser útil en algunos escenarios, pero

el principal puede ser el no saber qué objeto vamos a instanciar hasta el momento de la ejecución.



la instancia *iMotor* nose de que tipo es, hasta el momento de ejecución, donde le paso el tipo de motor que quiero crear mediante el parámetros tipo *Motor*.

Patrón de diseño Template Method: tiene que ver con el uso de las clases abstractas y la delegación en el hijo en ciertas responsabilidades que el padre no puede resolver, ya que depende de la especificación del hijo, en como se comportara.

Objetivos: permitir que ciertos pasos de un algoritmo definidos en un método de una clase sean redefinidos en sus clases derivadas sin necesidad de sobrecargar la operación entera. Si el patrón command nos permite encapsular una invocación a un método, el patrón template method o método modelo establece una forma de encapsular algoritmos.

Este patrón se basa en un principio: si un algoritmo puede aplicarse a varios supuestos en los que únicamente cambie un pequeño número de operaciones, la idea sería utilizar una clase para moldearlo a través de sus operaciones. Esta clase base se encargara de definir los pasos comunes del algoritmo, mientras que las clases que hereden de ella implementaran los detalles propios de cada caso concreto, es decir, el código específico para cada caso.

Procedimiento:

- Se declara una clase abstracta, que será la plantilla o modelo. Esta clase definira una serie de metodos, Aquellos que sean comunes estarán implementados. Aquellos que dependen de cada caso concreto, se declaran como **abstractos** obligando a las clases hijas a implementarlos.
- Cada clase derivada implementa los metodos especificos, acudiendo a la clase base para ejecutar el código común.
- La clase base también se encargará de la **lógica del algoritmo**, ejecutando los pasos en un orden preestablecido (las clases hijas, no deberían poder modificar el algoritmo, únicamente definir la funcionalidad específica que tiene que implementar).

Inversión de dependencias: la clase base no tiene por que saber nada sobre sus hijas, pero aun asi se encargará de invocar su funcionalidad (métodos) cuando sea necesario.

Algoritmos con gancho: se implementan en la clase base con un comportamiento por defecto, que puede ser sobrecargado en las clases derivadas o ejecutado como está por herencia directa. De este modo, se proporcionan pasos opcionales, haciendo que las clases derivadas enganchen funcionalidad opcional en ciertos puntos del algoritmo. Estos hooks suelen utilizar normalmente con cláusulas condicionales, de forma que se ejecuten en ciertas condiciones concretas.

¿Cuándo utilizar el patrón Template Method?: este patrón es aconsejable en los siguientes supuestos

- cuando se cuenta con un algoritmo aplicable a varias situaciones, cuya implementación difiere únicamente en algunos pasos
- arquitecturas donde los pasos de un proceso están definidos (el qué), pero sea necesario establecer los detalles sobre cómo realizarlos (el cómo)
- módulo en los que exista una gran cantidad de código duplicado que pueda ser factorizado en pasos comunes.